

技术支持工程师 · 面试题库

广州云趣信息科技有限公司

(超详细版 · 基础弱者也能看懂)

- ☑ 覆盖 JD 任职要求 二/三/四/五 所有条目
- ☑ 共 27+ 道题，每题都配：
 - ① ☑ 大白话原理解释
 - ② ☑ 面试作答模板
- ③ ☑☑ 速记口诀（考前 5 分钟过一遍）
- ☑ 最后附「高频命令速记表」

使用方法：

- 第一步：先读每道题的 ☑ 人话解释，真正理解原理；
- 第二步：背 ☑ 面试标准答案，面试时用自己的话说出来；
- 第三步：考前只看 ☑☑ 速记口诀，快速回忆；
- 第四步：进考场前翻最后一页的命令速记表。

二、数据库硬技能（JD 任职要求二）

☒☒ 本方向是 JD 权重最高部分，共 3 小点：MySQL/PostgreSQL 内核与调优、达梦数据库、Mycat/ProxySQL。

[2-1-1] 二.1 精通 MySQL 内核原理与性能调优

⚠☒ 为什么考这题：JD 第一条就是 MySQL 内核原理，面试官必问。

☒ 问题：MySQL 的 InnoDB 和 MyISAM 引擎到底有什么区别？为什么现在都用 InnoDB？

☒ 人话解释：把数据库想成一本笔记本：

- MyISAM 是一本写死了不能涂改的便签本——你可以很快翻看（读得快），但一旦写错就只能撕掉重写，而且没法保证别人看的和你写的是同一张纸。
- InnoDB 是一本可以涂改的活页本，还有对账功能（事务 ACID）。写的时候先在便签条上改（undo 日志），确认没问题再贴回去；如果写到一半笔没墨了，只要重新打开本子就能自动恢复（redo 日志 + 崩溃恢复）。
- 关键区别：InnoDB

支持事务、行级锁（多人可以同时写不同行）、外键、聚簇索引（数据和索引放一起）。5.5 以后 MySQL 默认用 InnoDB，因为实际业务都是既要读又要写。

把数据库想成一本笔记本：

- MyISAM 是一本写死了不能涂改的便签本——你可以很快翻看（读得快），但一旦写错就只能撕掉重写，而且没法保证别人看的和你写的是同一张纸。
- InnoDB 是一本可以涂改的活页本，还有对账功能（事务 ACID）。写的时候先在便签条上改（undo 日志），确认没问题再贴回去；如果写到一半笔没墨了，只要重新打开本子就能自动恢复（redo 日志 + 崩溃恢复）。
- 关键区别：InnoDB

支持事务、行级锁（多人可以同时写不同行）、外键、聚簇索引（数据和索引放一起）。5.5 以后 MySQL 默认用 InnoDB，因为实际业务都是既要读又要写。

☒ 面试标准答案：

InnoDB 和 MyISAM 的核心区别有 4 个维度：

- ① 事务：InnoDB 支持 ACID 事务（原子性/一致性/隔离性/持久性），靠 redo+undo 日志实现崩溃恢复；MyISAM 没有事务。
- ② 锁粒度：InnoDB 是行级锁，并发写入性能高；MyISAM 是表锁，一次只能一个人写整张表。
- ③ 索引结构：InnoDB 是聚簇索引（B+ 树，叶子节点存整行数据）；MyISAM 是非聚簇（索引和数据分开存）。
- ④ 其它：InnoDB 支持外键、MVCC 多版本并发控制；MyISAM 读极快但写不行，早已淘汰。现在生产环境 100% 用 InnoDB。

☒☒ 速记口诀：别忘提 redo/undo 日志和 MVCC，这是 InnoDB 的灵魂。

[2-1-2] 二.1 MySQL 内核原理

⚠☒ 为什么考这题：索引是数据库调优的核心，不答这个等于没学过 MySQL。

☒ 问题：什么是聚簇索引？什么是二级索引？回表是什么鬼？怎么避免？

☒ 人话解释：想象一个新华字典：

- 聚簇索引就是字典正文本身——按拼音排序，翻开某一页直接能看到这个字的全部信息（部首、释义、组词）。主键索引就是聚簇索引。
- 二级索引就是字典前面的部首检字表——上面只写着「木部 → 第 123 页」，你查到页码后还要翻到正文那一页才能看到完整内容。这「再翻一次正文」的动作就是回表。
- 怎么避免回表？用覆盖索引——就是让你查的所有字段都在二级索引里，直接在「检字表」上就能看到，不用再翻正文。

想象一个新华字典：

- 聚簇索引就是字典正文本身——按拼音排序，翻开某一页直接能看到这个字的全部信息（部首、释义、组词）。主键索引就是聚簇索引。
- 二级索引就是字典前面的部首检字表——上面只写着「木部 → 第 123 页」，你查到页码后还要翻到正文那一页才能看到完整内容。这「再翻一次正文」的动作就是回表。
- 怎么避免回表？用覆盖索引——就是让你查的所有字段都在二级索引里，直接在「检字表」上就能看到，不用再翻正文。

☒ 面试标准答案：

聚簇索引的叶子节点存整行完整数据，主键索引一定是聚簇索引；二级索引（普通索引）叶子节点存的是主键值，查到主键后要再去聚簇索引里取完整行，这个过程叫回表。

避免回表的方法叫覆盖索引：查询的所有字段都被包含在索引中（SELECT 字段 ⊆ 索引字段），MySQL 直接从索引就能返回结果，不需要回表。实际调优中要避免 SELECT *，只查需要的字段就是为了更容易命中覆盖索引。

☒☒ 速记口诀：一句话：覆盖索引 = 查什么就把什么都建进索引。

[2-1-3] 2.1 性能调优

△☒ 为什么考这题：JD 要「性能调优」，慢查询调优是第一考点。

☒ 问题：慢查询怎么排查？说完整的排查流程。

☒ 人话解释：这就像你家水管堵了：

- ① 先装一个堵点记录仪（开启 `slow_query_log`），记录每次通水超过 2 秒的情况；
- ② 翻记录仪看是哪根水管堵了（查看慢查询日志文件，或者用 `pt-query-digest` 这个专业工具分析）；
- ③ 找到堵的那根管子后，用 `EXPLAIN` 这个「内窥镜」伸进去看——是管子本身太细（没建索引），还是有东西卡住了（索引失效），还是管子拐了太多弯（SQL 写法问题）；
- ④ 根据内窥镜结果对症下药：加索引、改 SQL、分库分表。

这就像你家水管堵了：

- ① 先装一个堵点记录仪（开启 `slow_query_log`），记录每次通水超过 2 秒的情况；
- ② 翻记录仪看是哪根水管堵了（查看慢查询日志文件，或者用 `pt-query-digest` 这个专业工具分析）；
- ③ 找到堵的那根管子后，用 `EXPLAIN` 这个「内窥镜」伸进去看——是管子本身太细（没建索引），还是有东西卡住了（索引失效），还是管子拐了太多弯（SQL 写法问题）；
- ④ 根据内窥镜结果对症下药：加索引、改 SQL、分库分表。

☒ 面试标准答案：

慢查询排查完整流程：

- ① 开启慢查询日志：`slow_query_log=ON`，`long_query_time=1`（超过 1 秒就记录），指定

slow_query_log_file 路径。

② 分析日志：用 pt-query-digest (Percona 工具) 对日志做聚合分析，看哪种 SQL 最常见、哪种最慢。

③ EXPLAIN 分析：对单条慢查询用 EXPLAIN 看 type (至少 range, 最好 ref/eq_ref)、key (实际用到的索引)、rows (预估扫描行数, 越小越好)、Extra (Using filesort=排序没收敛到索引)。

④ 常见问题：没加索引、索引失效 (对索引列用函数、隐式类型转换、OR、范围条件后列不再生效)、LIMIT 大分页 (LIMIT 1000000,10 → 延迟关联/游标方案)、大表全表扫。

⑤ 优化手段：加合适的联合索引、改写 SQL、分库分表、读写分离。

☒☒ 速记口诀：EXPLAIN 一定要提，这是调优的必备工具。type 列的优先级：system > const > eq_ref > ref > range > index > ALL。

[2-1-4] 二.1 MGR 高可用集群

⚠☒ 为什么考这题：JD 明确提了 MGR，是核心加分项，面试官大概率会追问。

☒ 问题：MGR (MySQL Group Replication) 是什么？怎么部署？单主和多主有什么区别？

☒ 人话解释：想象一个三人投票小组 (节点数必须是奇数，因为要投票)：

- 大家共享一本账本 (数据)，任何人写东西都要广播给另外两个人，至少 2 个人同意才能写上去 (这就是 Paxos 一致性协议)。
- 单主模式：只有组长 (PRIMARY) 能写，另外两个组员 (SECONDARY) 只能读，但组长病倒了组员自动投票选新组长。这是生产最常用的。
- 多主模式：大家都能写，但如果你写了一行我也写了一行 (写冲突)，系统会自动回滚其中一个。一般不推荐，除非你很清楚自己在干什么。
- 部署就 5 步：装 MySQL 8.0 → 开 GTID 和 binlog_format=ROW → 三个节点互相配白名单 → 每台机器 START GROUP_REPLICATION → 看 performance_schema.replication_group_members 确认都 ONLINE。

想象一个三人投票小组 (节点数必须是奇数，因为要投票)：

- 大家共享一本账本 (数据)，任何人写东西都要广播给另外两个人，至少 2 个人同意才能写上去 (这就是 Paxos 一致性协议)。
- 单主模式：只有组长 (PRIMARY) 能写，另外两个组员 (SECONDARY) 只能读，但组长病倒了组员自动投票选新组长。这是生产最常用的。
- 多主模式：大家都能写，但如果你写了一行我也写了一行 (写冲突)，系统会自动回滚其中一个。一般不推荐，除非你很清楚自己在干什么。
- 部署就 5 步：装 MySQL 8.0 → 开 GTID 和 binlog_format=ROW → 三个节点互相配白名单 → 每台机器 START GROUP_REPLICATION → 看 performance_schema.replication_group_members 确认都 ONLINE。

☒ 面试标准答案：

MGR 是 MySQL 8.0 内置的高可用集群方案，基于 Paxos 分布式一致性协议实现。核心思想：事务提交前广播给所有节点，多数派同意才提交。

单主模式：1 PRIMARY (可读写) + N SECONDARY (只读)，最稳定，生产默认。PRIMARY 挂了自动选新 PRIMARY。

多主模式：所有节点可读写，冲突自动回滚，一般不用。

部署要点：奇数节点（3 或 5）、同机房低延迟、开 GTID+ROW

格式+binlog、group_replication_start_on_boot=OFF、bootstrap 设首个节点、组名称 UUID 一致、IP 白名单互配。

☒☒ 速记口诀：面试时主动说「我会部署 3 节点 MGR 单主集群」，再把部署 5 步说出来，直接加分。

[2-1-5] 二.1 PostgreSQL

△☒ 为什么考这题：JD 同时要求 MySQL 和 PostgreSQL，说明公司可能两套都在用。

☒ 问题：PostgreSQL 和 MySQL 到底啥不一样？什么时候该选 PG？

☒ 人话解释：把它们想成两个装修队：

- MySQL 装修队（InnoDB）：速度快、效率高，适合搞标准的出租房（典型 OLTP，简单增删改查），但装复杂的智能家居（复杂函数、半结构化数据）就有点费劲。
 - PG 装修队：手艺更精细，支持的花样更多——可以装地暖、智能马桶、全屋定制（JSONB、地理信息、递归查询、继承表、数组类型、CTE），而且 MVCC 是真多版本（不依赖 undo 日志），读的时候不阻塞写。但 PG 社区小，运维工具链没 MySQL 丰富。
- 选 PG 的典型场景：复杂 SQL 报表、地理数据（PostGIS）、半结构化 JSON、从 Oracle 迁移（PG 有 ORACLE 兼容模式）。

把它们想成两个装修队：

- MySQL 装修队（InnoDB）：速度快、效率高，适合搞标准的出租房（典型 OLTP，简单增删改查），但装复杂的智能家居（复杂函数、半结构化数据）就有点费劲。
 - PG 装修队：手艺更精细，支持的花样更多——可以装地暖、智能马桶、全屋定制（JSONB、地理信息、递归查询、继承表、数组类型、CTE），而且 MVCC 是真多版本（不依赖 undo 日志），读的时候不阻塞写。但 PG 社区小，运维工具链没 MySQL 丰富。
- 选 PG 的典型场景：复杂 SQL 报表、地理数据（PostGIS）、半结构化 JSON、从 Oracle 迁移（PG 有 ORACLE 兼容模式）。

☒ 面试标准答案：

PG 和 MySQL 的核心差异：

- ① MVCC 实现：PG 是真多版本（每条记录可能有多个可见版本，读不阻塞写）；MySQL InnoDB 是 undo 日志回滚实现的多版本。
 - ② 数据类型：PG 支持 JSONB（可索引的 JSON）、数组、范围类型、继承表、自定义类型；MySQL 类型相对简单。
 - ③ SQL 标准：PG 更标准，支持 CTE 递归、窗口函数、物化视图；MySQL 8.0 也加了但不如 PG 全。
 - ④ 适用场景：PG 适合复杂查询/OLAP/地理信息（PostGIS）/半结构化；MySQL 适合高并发简单读写（OLTP）、互联网应用。
- 公司同时用两者一般是：OLTP 核心用 MySQL，报表/数据分析用 PG。

☒☒ 速记口诀：别硬背，面试时说「PG 更像一个增强版的 Oracle，功能丰富但社区小」就行。

[2-2-1] 二.2 达梦数据库

△☒ 为什么考这题：JD 明确提了达梦，这是信创（国产化替代）时代的产物，必须答。

☒ 问题：达梦数据库是什么？为什么政府/国企项目要用它？你怎么运维它？

☒ 人话解释：信创就是「信息技术应用创新」——简单说就是国内政府/国企不能再用 Oracle/MySQL 这些外国数据库了，必须换国产的。达梦（DM）就是国产数据库里做了 20 年的老牌选手，武汉华工出的。

- 达梦的语法尽量兼容 Oracle（所以 Oracle 老鸟上手快），也支持部分 MySQL 语法；
- 运维工具：dminit 初始化、dmserver 服务启动、dmrman 备份恢复、DTS 数据迁移、DMETL ETL 工具；
- 集群：DMDS（分布式集群）、主备（类似 MySQL 主从）、MPP（大规模并行处理）。

面试时即使没真用过也没关系，说「它原理和 PG/Oracle 同源，我上手一周就能接管」就够了。

信创就是「信息技术应用创新」——简单说就是国内政府/国企不能再用 Oracle/MySQL 这些外国数据库了，必须换国产的。达梦（DM）就是国产数据库里做了 20 年的老牌选手，武汉华工出的。

- 达梦的语法尽量兼容 Oracle（所以 Oracle 老鸟上手快），也支持部分 MySQL 语法；
- 运维工具：dminit 初始化、dmserver 服务启动、dmrman 备份恢复、DTS 数据迁移、DMETL ETL 工具；
- 集群：DMDS（分布式集群）、主备（类似 MySQL 主从）、MPP（大规模并行处理）。

面试时即使没真用过也没关系，说「它原理和 PG/Oracle 同源，我上手一周就能接管」就够了。

☒ 面试标准答案：

达梦（DM）是国内最早的国产数据库厂商之一，武汉华工达梦，主打「信创」领域——政府、金融、能源等安全要求高的行业必须用国产数据库替代 Oracle/MySQL。

体系架构：单实例 → 主备集群（故障切换） → DMDS 分布式集群 → MPP 大规模并行。

运维动作：

- ① 初始化：dminit 命令指定端口、页大小、字符集；
- ② 注册服务：dm_service_installer.sh；
- ③ 备份：dmrman 工具支持联机备份（归档模式开着）和脱机备份；
- ④ 迁移：DTS（达梦迁移工具）从 Oracle/MySQL/SQL Server 一键迁。

实际运维和 PG/Oracle 高度相似，原理通了上手很快。

☒☒ 速记口诀：核心关键词：信创、国产化、Oracle 兼容、DTS 迁移、dmrman 备份。

[2-3-1] 二.3 Mycat、ProxySQL

△☒ 为什么考这题：JD 明确要「熟练使用 Mycat、ProxySQL」，而且提了「百 TB 级数据量」。

☒ 问题：Mycat 和 ProxySQL 分别是什么？做什么用的？分库分表又是啥？

☒ 人话解释：把它们想成餐厅的服务员+调度中心：

- 你的数据库集群像厨房，有很多大厨（MySQL 节点），每个大厨只管 10 万道菜（分片/分表）；
- Mycat / ProxySQL 就是门口的调度员——客人（你的应用程序）过来点菜（发 SQL），调度员看菜名（分片键，比如用户 ID 的前 2 位），决定送到哪个大厨那里；
- 读写分离就是：点菜（写请求）只找主厨（主库），看菜单（读请求）找任何一个帮厨（从库）；
- 分库分表就是：一桌菜（一张大表）做不完，拆到几个厨房（几个库/表）一起做；
- 区别：ProxySQL 是轻量高性能的代理，配置灵活，社区活跃；Mycat 是老牌中间件，功能多但社区维护一般，SQL 兼容性有坑。

把它们想成餐厅的服务员+调度中心：

- 你的数据库集群像厨房，有很多大厨（MySQL 节点），每个大厨只管 10 万道菜（分片/分表）；
- Mycat / ProxySQL 就是门口的调度员——客人（你的应用程序）过来点菜（发 SQL），调度员看菜名（分片键，比如用户 ID 的前 2 位），决定送到哪个大厨那里；

- 读写分离就是：点菜（写请求）只找主厨（主库），看菜单（读请求）找任何一个帮厨（从库）；
- 分库分表就是：一桌菜（一张大表）做不完，拆到几个厨房（几个库/表）一起做；
- 区别：ProxySQL 是轻量高性能的代理，配置灵活，社区活跃；Mycat 是老牌中间件，功能多但社区维护一般，SQL 兼容性有坑。

☒ 面试标准答案：

Mycat 和 ProxySQL 都是 MySQL 协议的中间件/代理层，部署在应用和 MySQL 集群之间。

核心功能：

- ① 读写分离（读走从库、写走主库）；
- ② 分库分表（大表拆到多个 MySQL 节点）；
- ③ 连接池管理（减少 MySQL 连接开销）；
- ④ SQL 缓存（热点查询直接返回）。

ProxySQL vs Mycat：ProxySQL 用 C++ 写，性能好、配置灵活（SQLite 存配置）、社区活跃；Mycat 是 Java 写的老牌产品，功能多但 SQL 兼容性问题多、更新慢。新架构推荐 ProxySQL + ShardingSphere。

分库分表的关键：选对分片键（高频查询条件 + 均匀分布 + 避免跨库 JOIN），常见策略 hash（取模）/range（按时间）/list（按地域）。

☒☒ 速记口诀：面试时举例子：比如用户表按 user_id 取模分到 4 个库，查 user_id=123 就直接路由到 $(123\%4=3)$ 第 3 个库。

三、云原生与虚拟化（JD 任职要求三）

☒☒ 3 小点：Docker 构建、Kubernetes、VMware/OpenStack。

[3-1-1] 3.1 Docker 镜像构建

⚠☒ 为什么考这题：JD 第一条就是 Docker，必考。

☒ 问题：Docker 到底是什么？镜像怎么构建？Dockerfile 怎么写才规范？

☒ 人话解释：Docker = 标准化的集装箱：

- 以前你部署一个应用，像搬家一样——你得自己打包行李（安装依赖）、租货车（装运行环境）、搬到公司现场（配置服务器）、卸货（配置路径），换一台机器又要重搬一遍。

- Docker 把整个家（应用+依赖+环境）塞进一个集装箱（镜像）里，到哪都能直接搬下来用，不管对方是笔记本还是云服务器。

- 怎么构建？写一个 Dockerfile（就像搬家用的打包清单）：

FROM 基础镜像（比如 python:3.10）→ COPY 依赖文件（先 requirements.txt）→ RUN 安装（pip install）→ COPY 源码（后 src）→ CMD 指定启动命令。

- 规范：用多阶段构建减小体积（比如第一阶段装编译工具，第二阶段只放编译产物）、依赖先装源码后拷（利用缓存，代码改了不用重装依赖）、加 HEALTHCHECK、别用 root 用户跑。

Docker = 标准化的集装箱：

- 以前你部署一个应用，像搬家一样——你得自己打包行李（安装依赖）、租货车（装运行环境）、搬到公司现场（配置服务器）、卸货（配置路径），换一台机器又要重搬一遍。

- Docker 把整个家（应用+依赖+环境）塞进一个集装箱（镜像）里，到哪都能直接搬下来用，不管对方是笔记本还是云服务器。

- 怎么构建？写一个 Dockerfile（就像搬家用的打包清单）：

FROM 基础镜像（比如 python:3.10）→ COPY 依赖文件（先 requirements.txt）→ RUN 安装（pip install）→ COPY 源码（后 src）→ CMD 指定启动命令。

- 规范：用多阶段构建减小体积（比如第一阶段装编译工具，第二阶段只放编译产物）、依赖先装源码后拷（利用缓存，代码改了不用重装依赖）、加 HEALTHCHECK、别用 root 用户跑。

☒ 面试标准答案：

Docker 是容器化技术，核心思想「一次构建、到处运行」。

镜像构建流程：Dockerfile 写指令 → docker build 逐层构建 → 每层都是缓存 → 最终生成一个可运行的镜像。

Dockerfile 最佳实践：

- ① FROM 选最小基础镜像（alpine/debian-slim）；
- ② 合并 RUN 指令减少层数；
- ③ 先 COPY 依赖文件再 COPY 源码（利用缓存，依赖不变时无需重装）；
- ④ 用多阶段构建（第一阶段编译、第二阶段只拷贝产物）；
- ⑤ 指定非 root 用户运行；
- ⑥ 加 HEALTHCHECK 探针；
- ⑦ .dockerignore 排除无用文件。

☒☒ 速记口诀：多阶段构建是减体积的必杀技，面试一定要提。

[3-1-2] 3.1 Docker Compose

⚠☒ 为什么考这题：JD 同时要求 Compose 和 K8s，面试官可能对比着问。

☒ 问题：Docker Compose 是啥？和 K8s 有啥关系？什么时候用哪个？

☒ 人话解释：把它们想成搬家的两种方案：

- Docker Compose：你自己搬几间房的家（单机多容器），一张清单（docker-compose.yml）写清楚几个集装箱、谁先装、谁需要谁（比如 web 依赖 db），一键 docker-compose up 搞定。适合开发环境、测试环境、单机小项目。

- Kubernetes (K8s)：搬家公司搬整栋楼的家（集群级管理），有调度系统（自动把集装箱分配到不同楼层）、有保安（健康检查自动替换坏集装箱）、有调度员（自动伸缩人多了加集装箱、人少了减）。适合生产环境、多节点、高可用需求。

关系：Compose 适合起步 → 规模大了迁移到 K8s。Docker 官方也做了 Compose on Kubernetes 的桥梁。

把它们想成搬家的两种方案：

- Docker Compose：你自己搬几间房的家（单机多容器），一张清单（docker-compose.yml）写清楚几个集装箱、谁先装、谁需要谁（比如 web 依赖 db），一键 docker-compose up 搞定。适合开发环境、测试环境、单机小项目。

- Kubernetes (K8s)：搬家公司搬整栋楼的家（集群级管理），有调度系统（自动把集装箱分配到不同楼层）、有保安（健康检查自动替换坏集装箱）、有调度员（自动伸缩人多了加集装箱、人少了减）。适合生产环境、多节点、高可用需求。

关系：Compose 适合起步 → 规模大了迁移到 K8s。Docker 官方也做了 Compose on Kubernetes 的桥梁。

☒ 面试标准答案：

Docker Compose 是单机多容器编排工具，用 YAML

文件定义多个服务的启动顺序、依赖关系、网络和存储，一键启动整个应用栈。

K8s 是集群级容器编排平台，核心概念：Pod（最小调度单元，一个或多个容器）、Service（服务发现+负载均衡）、Deployment（滚动更新/自动伸缩）、ConfigMap/Secret（配置）、Ingress（入口路由）、PV/PVC（持久化存储）。

选型：单节点且不复杂用 Compose（开发/测试/小型生产）；多节点/高可用/弹性伸缩必须

K8s（企业级生产）。Compose 文件可以用 kompose 工具转换成 K8s YAML。

☒☒ 速记口诀：一句话总结：Compose=单机编排；K8s=集群编排。

[3-2-1] 三.2 Kubernetes

△☒ 为什么考这题：JD 写了「熟悉 K8s 技术栈」，不用太深入但得知道骨架。

☒ 问题：K8s 的核心组件有哪些？Pod 挂了怎么排查？

☒ 人话解释：K8s 像一个城市管理系统：

- 控制面（市政府）：
 - kube-apiserver（政务大厅，所有命令都先到这登记）；
 - etcd（市政档案库，存所有状态）；
 - kube-scheduler（城管调度员，决定哪个小区放哪个集装箱）；
 - kube-controller-manager（各部门局长，负责运维 Pod 生命周期）。

- 数据面（小区）：
 - kubelet（小区物业，实际管理容器生命周期）；
 - kube-proxy（小区保安，管网络转发/负载均衡）；
 - containerd/docker（集装箱码头，真正跑容器）。

- Pod CrashLoopBackOff（集装箱打不开门）排查流程：

kubectl describe pod 看 Events → kubectl logs --previous 看上一次崩溃日志 → 进 Pod 用 sh

手动跑命令看报啥错 → 常见原因：启动命令错、探针太严、端口冲突、OOMKilled、配置文件缺失。

K8s 像一个城市管理系统：

- 控制面（市政府）：
 - kube-apiserver（政务大厅，所有命令都先到这登记）；
 - etcd（市政档案库，存所有状态）；
 - kube-scheduler（城管调度员，决定哪个小区放哪个集装箱）；
 - kube-controller-manager（各部门局长，负责运维 Pod 生命周期）。
- 数据面（小区）：
 - kubelet（小区物业，实际管理容器生命周期）；
 - kube-proxy（小区保安，管网络转发/负载均衡）；
 - containerd/docker（集装箱码头，真正跑容器）。
- Pod CrashLoopBackOff（集装箱打不开门）排查流程：

kubectl describe pod 看 Events → kubectl logs --previous 看上一次崩溃日志 → 进 Pod 用 sh

手动跑命令看报啥错 →

常见原因：启动命令错、探针太严、端口冲突、OOMKilled、配置文件缺失。

☒ 面试标准答案：

K8s 架构：

控制面：kube-apiserver（统一入口）+ etcd（分布式 KV

存储，集群状态）+ kube-scheduler（调度决策）+ kube-controller-manager（控制器循环）

数据面：kubelet（节点代理，管理 Pod）+ kube-proxy（网络/SLB）+ 容器运行时（containerd）

排查步骤（Pod CrashLoopBackOff）：

- ① kubectl describe pod → 看 Events 字段，找 FailedScheduling/Liveness probe failed 等；
- ② kubectl logs -c --previous → 看上一次崩溃的日志，最关键；
- ③ kubectl exec -it -- sh → 进入容器手动执行启动命令复现；
- ④ 常见根因：command/args 配置错、ConfigMap/Secret 没挂载、OOMKilled（资源不够）、Liveness 探针超时被反复杀、依赖的 DB/Redis 没起来。

☒☒ 速记口诀：kubectl 三板斧：get / describe / logs，面试时说这三个最常用。

[3-3-1] 3.3 VMware / KVM / OpenStack

⚠☒ 为什么考这题：JD 要「熟悉 VMware vSphere 或 KVM/OpenStack」，得说出定位差异。

☒ 问题：VMware、KVM、OpenStack 三者啥关系？企业私有云选哪个？

☒ 人话解释：把它们想成盖房子的三种方案：

- KVM = 一块砖（Linux 内核里的虚拟化模块，最底层）；
 - VMware vSphere = 一整套预制别墅（商业闭源、ESXi 作为 hypervisor + vCenter 集中管理，成熟稳定但要付费）；
 - OpenStack = 开放的建筑工地平台（开源，Nova 管计算 / Cinder 管存储 / Neutron 管网络 / Glance 管镜像 / Keystone 管认证，可以用 KVM 作为底层虚拟化）。
- 关系：OpenStack 底层可以用 KVM；VMware 是独立体系。企业选哪个？有钱要省心选 VMware；要灵活/要国产化选 OpenStack + KVM。

把它们想成盖房子的三种方案：

- KVM = 一块砖（Linux 内核里的虚拟化模块，最底层）；

- VMware vSphere = 一整套预制别墅（商业闭源、ESXi 作为 hypervisor + vCenter 集中管理，成熟稳定但要付费）；
 - OpenStack = 开放的建筑工地平台（开源，Nova 管计算 / Cinder 管存储 / Neutron 管网络 / Glance 管镜像 / Keystone 管认证，可以用 KVM 作为底层虚拟化）。
- 关系：OpenStack 底层可以用 KVM；VMware 是独立体系。企业选哪个？有钱要省心选 VMware；要灵活/要国产化选 OpenStack + KVM。

☑ 面试标准答案：

定位差异：

- KVM（Kernel-based Virtual Machine）：Linux 内核自带的 hypervisor，最底层的虚拟化能力；
 - VMware vSphere：商业虚拟化平台（ESXi 作为 hypervisor + vCenter 集中管理），成熟稳定、生态好、付费有支持，但厂商锁定；
 - OpenStack：开源 IaaS 云平台，由 Nova(计算)/Cinder(块存储)/Neutron(网络)/Glance(镜像)/Keystone(认证) 等组件组成，底层可以接 KVM/QEMU/LXC。
- 选型：国企/政府信创要求必须 OpenStack；有钱的传统企业用 vSphere 省心；技术团队强的公司自建 OpenStack 也 OK。现在趋势是 OpenStack + K8s 融合。

☑☑ 速记口诀：记清三者定位：KVM 是底座，VMware 是商业产品，OpenStack 是开源云平台。

四、监控与自动化（JD 任职要求四）

☒☒ 2 小点：Zabbix/Prometheus/Grafana、Shell 脚本/Python。

[4-1-1] 四.1 Zabbix / Prometheus / Grafana

△☒ 为什么考这题：JD 写了「至少两种」，三个工具的定位必须分清。

☒ 问题：Zabbix、Prometheus、Grafana 三个都是啥？分别做什么？哪个好？

☒ 人话解释：把它们想成医院体检的三位医生：

- Zabbix = 老派内科医生，拿听诊器（Agent 主动上报），开体检表（监控模板），记录数据（MySQL 存），哪个指标超了就报警（Trigger + MediaType）。适合传统服务器/物理机，模板丰富，配好就能用。

- Prometheus = 现代全科医生，主动问（Pull 模型，定期去各 /metrics 端口拉数据），用新设备（多维度标签、PromQL 强大查询语言），数据存自己的 TSDB（时序数据库），适合云原生/K8s。

- Grafana = 体检报告可视化，把 Zabbix/Prometheus 的数据用漂亮的仪表盘画出来，不是监控系统本身，是展示层。

选哪个？有 K8s/微服务用 Prometheus 全家桶；传统 VM 用 Zabbix。Grafana 两个都能配。

把它们想成医院体检的三位医生：

- Zabbix = 老派内科医生，拿听诊器（Agent 主动上报），开体检表（监控模板），记录数据（MySQL 存），哪个指标超了就报警（Trigger + MediaType）。适合传统服务器/物理机，模板丰富，配好就能用。

- Prometheus = 现代全科医生，主动问（Pull 模型，定期去各 /metrics 端口拉数据），用新设备（多维度标签、PromQL 强大查询语言），数据存自己的 TSDB（时序数据库），适合云原生/K8s。

- Grafana = 体检报告可视化，把 Zabbix/Prometheus 的数据用漂亮的仪表盘画出来，不是监控系统本身，是展示层。

选哪个？有 K8s/微服务用 Prometheus 全家桶；传统 VM 用 Zabbix。Grafana 两个都能配。

☒ 面试标准答案：

定位分工：

- Zabbix：传统监控，Agent 推送 + 无 Agent 拉取，MySQL 存数据，模板丰富，适合基础设施监控；

- Prometheus：云原生监控，Pull 模型（HTTP 拉各 target 的 /metrics 端点）+ 多维度标签 + PromQL 查询 + 内置告警规则，适合动态 K8s 环境；

- Grafana：数据可视化面板，接 Prometheus/Zabbix/ELK 等各种数据源，画图表/仪表盘。

结论：JD 说至少两种 → 选 Prometheus + Grafana（最流行组合），再补充 Zabbix 也行。生产常见组合：Prometheus + Alertmanager（告警路由）+ Grafana。

☒☒ 速记口诀：Prometheus 核心三要素：Pull 模型、多维度 label、PromQL。

[4-2-1] 四.2 Shell 脚本

△☒ 为什么考这题：JD 明确要「Shell 脚本功底扎实」，这是经典考题。

☒ 问题：写一个每天凌晨 2 点自动备份 MySQL 并保留 7 天的 Shell 脚本，然后解释每一行。

☒ 人话解释：这题考的就是三件事：mysqldump

怎么用、怎么定时跑（crontab）、怎么清理旧备份。每行代码的人话解释：

、、、

#!/bin/bash → 告诉系统用 bash 解释脚本

BACKUP_DIR=/data/backup → 备份文件放哪

DB_USER=backup → 备份专用账号（别用 root）

DB_PASS='xxx' → 密码（实际应该存 .my.cnf）

DATE=\$(date +%F_%H%M) → 时间戳，文件名用，例如 2026-10-09_0200

mkdir -p \$BACKUP_DIR → 目录不存在就建

mysqldump -u\$DB_USER -p\$DB_PASS \ → 开始备份

--all-databases \ → 备份所有库（或者只写具体库名）

--single-transaction \ → InnoDB 一致性快照，不锁表

--routines --triggers \ → 连存储过程/触发器一起备份

| gzip > \$BACKUP_DIR/all_\$DATE.sql.gz → 压缩存成 .gz 文件省空间

find \$BACKUP_DIR -name '*.sql.gz' -mtime +7 -delete → 超过 7 天的删了

、、、

然后用 crontab -e 加一行定时任务：

`0 2 \* \* \* /opt/scripts/mysql\_backup.sh >> /var/log/backup.log 2>&1`

意思是：每天（* * *）凌晨 2 点（0 2）跑这个脚本，标准输出和错误都写日志。

这题考的就是三件事：mysqldump

怎么用、怎么定时跑（crontab）、怎么清理旧备份。每行代码的人话解释：

、、、

#!/bin/bash → 告诉系统用 bash 解释脚本

BACKUP_DIR=/data/backup → 备份文件放哪

DB_USER=backup → 备份专用账号（别用 root）

DB_PASS='xxx' → 密码（实际应该存 .my.cnf）

DATE=\$(date +%F_%H%M) → 时间戳，文件名用，例如 2026-10-09_0200

mkdir -p \$BACKUP_DIR → 目录不存在就建

mysqldump -u\$DB_USER -p\$DB_PASS \ → 开始备份

--all-databases \ → 备份所有库（或者只写具体库名）

--single-transaction \ → InnoDB 一致性快照，不锁表

--routines --triggers \ → 连存储过程/触发器一起备份

| gzip > \$BACKUP_DIR/all_\$DATE.sql.gz → 压缩存成 .gz 文件省空间

find \$BACKUP_DIR -name '*.sql.gz' -mtime +7 -delete → 超过 7 天的删了

、、、

然后用 crontab -e 加一行定时任务：

`0 2 \* \* \* /opt/scripts/mysql\_backup.sh >> /var/log/backup.log 2>&1`

意思是：每天（* * *）凌晨 2 点（0 2）跑这个脚本，标准输出和错误都写日志。

☒ 面试标准答案：

脚本如上。关键参数解释：

--single-transaction：最重要的参数，对 InnoDB 开启一致性快照，不锁表不影响业务；

-routines --triggers --events：备份存储过程/触发器/事件调度器，别漏；

| gzip：管道压缩，mysqldump 输出直接压缩写盘；

find -mtime +7 -delete：find 命令找超过 7×24h 的文件删除，-mtime +7 表示修改时间早于 7 天前。

生产改进：密码不写死（export MYSQL_PWD 或 .my.cnf + chmod 600）、备份后校验 MD5、备份文件同步到异地存储、加告警（备份失败发邮件）。

☒☒ 速记口诀：背一下 crontab 5 个星号的含义：分 时 日 月 周。

[4-2-2] 四.2 CPU 飙高排查

⚠☒ 为什么考这题：运维日常高频场景，面试必问排障思路。

☒ 问题：线上 CPU 突然飙到 100%，你怎么一步步定位根因？

☒ 人话解释：就像你家客厅突然特别吵（CPU 满了）：

- ① 先看是谁在吵（top -c / htop，按 P 键按 CPU 排序），找到 PID；
- ② 看是这个进程里哪个线程在吵（pidstat -p PID -u -t，或者 top -H -p PID）；
- ③ 用 perf top 抓热点函数（相当于拿个偷听设备看它在说啥），perf record -g 抓堆栈火焰图；
- ④ 如果是 Java 进程：jstack -l PID 看线程栈，arthas（阿里开源诊断工具）在线 debug；
- ⑤ 看是用户态（业务代码）还是内核态（sy 列高 = 内核系统调用多，可能是 IO 风暴）；
- ⑥ 根源常见：死循环、GC 风暴（Java）、数据库慢 SQL 全表扫、中断风暴（硬件问题）、oom-killer 疯狂杀进程。

就像你家客厅突然特别吵（CPU 满了）：

- ① 先看是谁在吵（top -c / htop，按 P 键按 CPU 排序），找到 PID；
- ② 看是这个进程里哪个线程在吵（pidstat -p PID -u -t，或者 top -H -p PID）；
- ③ 用 perf top 抓热点函数（相当于拿个偷听设备看它在说啥），perf record -g 抓堆栈火焰图；
- ④ 如果是 Java 进程：jstack -l PID 看线程栈，arthas（阿里开源诊断工具）在线 debug；
- ⑤ 看是用户态（业务代码）还是内核态（sy 列高 = 内核系统调用多，可能是 IO 风暴）；
- ⑥ 根源常见：死循环、GC 风暴（Java）、数据库慢 SQL 全表扫、中断风暴（硬件问题）、oom-killer 疯狂杀进程。

☒ 面试标准答案：

完整排查步骤（由外及内）：

- ① top/htop：找 CPU% 最高的进程 PID，区分 us(用户态)/sy(内核态)/wa(IO 等待)/si(软中断)/st(虚拟化偷取)；
- ② pidstat -p PID -u -t：看该进程内哪个线程在烧 CPU，拿到线程 TID；
- ③ perf top / perf record -g sleep 10：抓热点函数调用栈，火焰图可视化；
- ④ Java 专项：top -H -p PID 拿线程 TID → printf '%x\n' TID 转 16 进制 → jstack PID | grep 16进制TID → 定位线程代码；或直接用 Arthas；
- ⑤ 辅助判断：iostat 看 IO 是否高 → vmstat 看上下文切换是否多 → sar -n DEV 看网络；
- ⑥ 常见根因：代码死循环、正则回溯风暴、GC 过度（堆太小）、数据库慢查询、网卡中断风暴、容器 CPU 限流触发 throttling。

☒☒ 速记口诀：关键工具链：top → pidstat → perf。Java 进程一定要提 jstack/arhas。

五、网络实操（JD 任职要求五）

☒☒ 2 小点：TCP/IP/LAN/STP/OSPF、H3C 设备配置、企业级组网。

[5-1-1] 五.1 TCP/IP

△☒ 为什么考这题：网络第一题，JD 第一条，背下来！

☒ 问题：TCP 三次握手到底是啥？为什么要三次不能两次？四次挥手的 TIME_WAIT 又是干嘛的？

☒ 人话解释：把 TCP 想象成打电话确认身份：

• 三次握手 = 拨号：

第一次：你拨出电话（客户端发 SYN=1）→「喂？在吗？」

第二次：对方接起来（服务端回 SYN+ACK）→「在！你是小明吧？」

第三次：你确认（客户端回 ACK）→「对我！」→通了，可以说话了。

为啥不能两次？因为服务端收到 SYN 之后不知道客户端能不能收到自己的声音，所以必须第三次 ACK 确认——双向都确认收发能力。

• 四次挥手 = 挂电话：TCP 是全双工的，你说完了对方可能还没说完，所以要分两次 FIN 分别关两个方向。

• TIME_WAIT = 挂电话后等 2 分钟：确保最后一个 ACK 一定能送达对方（不然对方重发 FIN），同时让残留的数据包在网络里消散，防止新连接收到旧数据。2 分钟 = 2MSL（报文最大存活时间）。

把 TCP 想象成打电话确认身份：

• 三次握手 = 拨号：

第一次：你拨出电话（客户端发 SYN=1）→「喂？在吗？」

第二次：对方接起来（服务端回 SYN+ACK）→「在！你是小明吧？」

第三次：你确认（客户端回 ACK）→「对我！」→通了，可以说话了。

为啥不能两次？因为服务端收到 SYN 之后不知道客户端能不能收到自己的声音，所以必须第三次 ACK 确认——双向都确认收发能力。

• 四次挥手 = 挂电话：TCP 是全双工的，你说完了对方可能还没说完，所以要分两次 FIN 分别关两个方向。

• TIME_WAIT = 挂电话后等 2 分钟：确保最后一个 ACK 一定能送达对方（不然对方重发 FIN），同时让残留的数据包在网络里消散，防止新连接收到旧数据。2 分钟 = 2MSL（报文最大存活时间）。

☒ 面试标准答案：

三次握手：SYN → SYN+ACK → ACK。双方各确认一次自己的发送和接收能力正常。为什么不能两次？两次握手服务端无法确认客户端是否能收到自己的报文（半开连接），会导致服务端白白维护无效连接。

四次挥手：由于 TCP

全双工，两个方向的连接必须分别关闭。FIN→ACK→FIN→ACK。如果一方收到 FIN 但还有数据要发，先发 ACK 不发 FIN。

TIME_WAIT（主动关闭方进入）：

① 确保最后一个 ACK 能被对方收到（丢了对对方会重发 FIN）；

② 让网络中残留的迟到报文消散，防止被新连接误收；

③ 2MSL（报文最长存活时间 × 2）。服务端大量 TIME_WAIT 时可以调 net.ipv4.tcp_tw_reuse。

☒☒ 速记口诀：面试时画个简单的时序图，加分。TIME_WAIT 一定要说 2MSL。

△☒ 为什么考这题：JD 提了 VLAN 和 H3C，企业网络高频考点。

☒ 问题：VLAN 是什么？Trunk 口和 Access 口有什么区别？H3C 怎么配？

☒ 人话解释：把交换机想成一个大教室，里面有 48 个插座（48 个物理端口）：

- 没有 VLAN 的时候，教室所有人讲话大家都能听见（广播域太大）。
- VLAN 就是用屏风把教室分成 10 个小组（10 个广播域），每组的讲话只有同组能听见。IP 网段必须和 VLAN 对应（VLAN 10 用 192.168.10.0/24，VLAN 20 用 192.168.20.0/24）。
- Access 口 = 只属于一个小组的插座（接终端 PC/服务器），不打 VLAN 标签；
- Trunk 口 = 多小组共用的走廊（接另一台交换机），带着 802.1Q 标签告诉对端「这个帧是 VLAN 10 的」。

H3C 配置示例：

、、、

system-view → 进入系统视图

vlan 100 → 创建 VLAN 100

port GigabitEthernet 1/0/1 → 把 1 口加进 VLAN（Access 模式默认）

interface GigabitEthernet 1/0/24

port link-type trunk → 24 口设为 Trunk

port trunk allow-pass vlan 100 200 → 允许 VLAN 100 和 200 通过

interface Vlan-interface 100 → 配 VLAN100 的网关

ip address 192.168.10.1 24

、、、

把交换机想成一个大教室，里面有 48 个插座（48 个物理端口）：

- 没有 VLAN 的时候，教室所有人讲话大家都能听见（广播域太大）。
- VLAN 就是用屏风把教室分成 10 个小组（10 个广播域），每组的讲话只有同组能听见。IP 网段必须和 VLAN 对应（VLAN 10 用 192.168.10.0/24，VLAN 20 用 192.168.20.0/24）。
- Access 口 = 只属于一个小组的插座（接终端 PC/服务器），不打 VLAN 标签；
- Trunk 口 = 多小组共用的走廊（接另一台交换机），带着 802.1Q 标签告诉对端「这个帧是 VLAN 10 的」。

H3C 配置示例：

、、、

system-view → 进入系统视图

vlan 100 → 创建 VLAN 100

port GigabitEthernet 1/0/1 → 把 1 口加进 VLAN（Access 模式默认）

interface GigabitEthernet 1/0/24

port link-type trunk → 24 口设为 Trunk

port trunk allow-pass vlan 100 200 → 允许 VLAN 100 和 200 通过

interface Vlan-interface 100 → 配 VLAN100 的网关

ip address 192.168.10.1 24

、、、

☒ 面试标准答案：

VLAN 原理：二层广播域隔离技术，通过在以太帧中插入 802.1Q Tag（4 字节）标记所属 VLAN，实现一台物理交换机模拟多台虚拟交换机。

Access vs Trunk：Access 口只属于 1 个 VLAN，收发不带标签；Trunk 口允许多 VLAN

通过，收带标签帧、发带标签帧（native VLAN 例外）。

H3C 配置如上。

生产注意：同一个 VLAN 跨交换机时两端 Trunk 必须都 allow 该 VLAN；VLAN 1 是默认 native VLAN，一般清空。

☒☒ 速记口诀：记关键词：802.1Q 标签、广播域隔离、Access=终端、Trunk=交换机互连。

[5-1-3] 5.1 STP

△☒ 为什么考这题：网络防环协议，JD 明确提了 STP。

☒ 问题：STP 是什么？根桥怎么选举？RSTP 和 STP 有啥区别？

☒ 人话解释：想象一个公司局域网，两台交换机都接了 2 根网线（冗余链路）：

- 如果不做任何处理，发一个广播包（比如 ARP 请求）会在两根网线之间无限循环（广播风暴），整个网络瞬间瘫痪。
- STP（生成树协议）就是自动找出一条最短路径，把其它冗余网线「假装拔掉」（端口阻塞）。如果最短路径断了，它会自动把备用线路接上来。
- 根桥选举：比谁的桥优先级(Priority + MAC) 最小——先比优先级（默认 32768，设小的当根），再比 MAC 地址。根桥是整个网络的中心。
- RSTP（快速生成树）改进 STP 三大缺点：STP 收敛要 50 秒，RSTP 只要 1-2 秒；STP 只有一个阻塞端口角色，RSTP 加了 Backup/Alternate；STP 没有端口主动转发机制，RSTP 有 P/A 协商。现在生产基本不用 STP，直接 RSTP 或 MSTP（多生成树，每个 VLAN 一棵生成树，负载均衡）。

想象一个公司局域网，两台交换机都接了 2 根网线（冗余链路）：

- 如果不做任何处理，发一个广播包（比如 ARP 请求）会在两根网线之间无限循环（广播风暴），整个网络瞬间瘫痪。
- STP（生成树协议）就是自动找出一条最短路径，把其它冗余网线「假装拔掉」（端口阻塞）。如果最短路径断了，它会自动把备用线路接上来。
- 根桥选举：比谁的桥优先级(Priority + MAC) 最小——先比优先级（默认 32768，设小的当根），再比 MAC 地址。根桥是整个网络的中心。
- RSTP（快速生成树）改进 STP 三大缺点：STP 收敛要 50 秒，RSTP 只要 1-2 秒；STP 只有一个阻塞端口角色，RSTP 加了 Backup/Alternate；STP 没有端口主动转发机制，RSTP 有 P/A 协商。现在生产基本不用 STP，直接 RSTP 或 MSTP（多生成树，每个 VLAN 一棵生成树，负载均衡）。

☒ 面试标准答案：

STP 是二层防环路协议，核心：阻塞冗余链路、生成一棵无环树。

根桥选举：Bridge ID = Priority(16bit, 可调) + MAC(48bit)，最小者成为根桥，每 2s 发一次 BPDU。

端口角色：Root Port（到根桥最近）、Designated Port（负责发 BPDU）、Blocking Port（阻塞，备用）。

STP vs RSTP：STP 收敛慢（50s: 20s 阻塞+15s 监听+15s 学习），RSTP 引入 P/A 协商和 edge port，收敛 1-2s；RSTP 增加 Alternate/Backup 端口角色；MSTP 支持多实例（每 VLAN 一棵树做负载均衡）。

企业网络现在只用 RSTP 或 MSTP，STP 只在老设备里出现。

☒☒ 速记口诀：根桥选举就记：优先级 → MAC，最小者胜。

[5-1-4] 5.1 OSPF

△☒ 为什么考这题：JD 提了 OSPF，企业网络的主力路由协议。

☒ 问题：OSPF 是什么？和 RIP 有啥区别？OSPF 为啥要分区域？

☒ 人话解释：想象两个快递公司：

- RIP：快递员每 30 秒给所有邻居发一张「我到每个城市要绕几圈」的清单（距离矢量，跳数 ≤ 15 ）。绕的圈数最多 15 圈，再多就不去了（16 跳不可达）。问题：收消息慢（收敛要几分钟）、容易形成环路、路走远了不知道走的是不是同一条路。已淘汰。
- OSPF：每个快递员都会画出整个城市的地图（链路状态数据库 LSDB），然后用 Dijkstra 最短路径算法算出自己到每个城市的最优路线。每个快递员都有相同的地图，所以永远不会绕路（无环路），而且消息传递很快（LSA 泛洪，收敛秒级）。
- 为啥分区域：城市太大了每个人都要记全地图太累（内存和 CPU 扛不住），所以把城市分成若干区（Area 0=市中心，Area 1/2=郊区），郊区的快递员只要记市中心怎么走，具体到郊区内部的路只有本区的快递员记得。Area 0 是骨干区域，所有区必须直接连 Area 0。

想象两个快递公司：

- RIP：快递员每 30 秒给所有邻居发一张「我到每个城市要绕几圈」的清单（距离矢量，跳数 ≤ 15 ）。绕的圈数最多 15 圈，再多就不去了（16 跳不可达）。问题：收消息慢（收敛要几分钟）、容易形成环路、路走远了不知道走的是不是同一条路。已淘汰。
- OSPF：每个快递员都会画出整个城市的地图（链路状态数据库 LSDB），然后用 Dijkstra 最短路径算法算出自己到每个城市的最优路线。每个快递员都有相同的地图，所以永远不会绕路（无环路），而且消息传递很快（LSA 泛洪，收敛秒级）。
- 为啥分区域：城市太大了每个人都要记全地图太累（内存和 CPU 扛不住），所以把城市分成若干区（Area 0=市中心，Area 1/2=郊区），郊区的快递员只要记市中心怎么走，具体到郊区内部的路只有本区的快递员记得。Area 0 是骨干区域，所有区必须直接连 Area 0。

☒ 面试标准答案：

OSPF（Open Shortest Path First）是链路状态路由协议，相比 RIP（距离矢量）：

- ① 收敛快（秒级 vs 分钟级）；
- ② 无环路（SPF 算法保证）；
- ③ 支持 VLSM（可变长子网掩码）；
- ④ Metric = 带宽的倒数（ $\text{cost} = 10^8 / \text{带宽}$ ）。

为什么分区域：OSPF 启动时要在全 LSA 上跑 SPF，网络越大越慢。分 Area 后，非骨干 Area 的内部拓扑 LSA 不会扩散到其它 Area，减小每个路由器的 LSDB 规模，降低 CPU/内存开销。Area 0 是骨干，必须其它 Area 直连 Area 0。

企业内网基本用 OSPF 或 ISIS，RIP 淘汰。

☒☒ 速记口诀：记关键词：链路状态、Dijkstra 算法、Area 0 骨干、SPF 无环。

[5-2-1] 五.2 H3C SSH 配置

△☒ 为什么考这题：JD 提了「能独立配置 H3C 网络设备」，SSH 配置是基础中的基础。

☒ 问题：怎么让一台全新的 H3C 交换机/路由器支持 SSH 登录？按步骤说。

☒ 人话解释：这题像给一台新手机装 APP 并设置密码锁，按顺序来：
、、、

1. 配管理 IP（给手机办手机号）

```
system-view  
interface Vlan-interface 1 → 默认 VLAN1 的三层接口  
ip address 192.168.1.100 255.255.255.0 → 管理 IP  
quit
```

2. 开 SSH 服务（安装 APP）

```
stelnet server enable → 开启 SSH（H3C 叫 stelnet）  
ssh server enable → 有些型号还要这条
```

3. 生密钥（给手机加密）

```
public-key local create rsa → 生成 RSA 密钥对（回车默认位数）  
public-key local create dsa → 有些型号也支持 DSA
```

4. 建登录账号（设密码锁）

```
local-user admin class manage  
password cipher Admin123 → 密码（cipher 是密文显示）  
service-type ssh terminal → 允许 SSH 和 Console 登录  
authorization-attribute user-role network-admin → 权限最高  
quit
```

5. 配 VTY（SSH 登录用虚拟终端）

```
user-interface vty 0 63  
authentication-mode scheme → 用 AAA 或 local 认证  
protocol inbound ssh → 只允许 SSH，禁 Telnet  
quit  
```
```

配完之后电脑 `ssh admin@192.168.1.100` 就能登上去了。PC 和交换机必须互通，防火墙不能挡 22 端口。

这题像给一台新手机装 APP 并设置密码锁，按顺序来：

````

1. 配管理 IP（给手机办手机号）

```
system-view  
interface Vlan-interface 1 → 默认 VLAN1 的三层接口  
ip address 192.168.1.100 255.255.255.0 → 管理 IP  
quit
```

2. 开 SSH 服务（安装 APP）

```
stelnet server enable → 开启 SSH（H3C 叫 stelnet）  
ssh server enable → 有些型号还要这条
```

3. 生密钥（给手机加密）

```
public-key local create rsa → 生成 RSA 密钥对（回车默认位数）  
public-key local create dsa → 有些型号也支持 DSA
```

4. 建登录账号（设密码锁）

local-user admin class manage

password cipher Admin123 → 密码（cipher 是密文显示）

service-type ssh terminal → 允许 SSH 和 Console 登录

authorization-attribute user-role network-admin → 权限最高

quit

5. 配 VTY（SSH 登录用虚拟终端）

user-interface vty 0 63

authentication-mode scheme → 用 AAA 或 local 认证

protocol inbound ssh → 只允许 SSH，禁 Telnet

quit

、、、

配完之后电脑 `ssh admin@192.168.1.100` 就能登上去了。PC 和交换机必须互通，防火墙不能挡 22 端口。

☑ 面试标准答案：

H3C SSH 配置五步：

① 配管理 IP：interface Vlan-interface X → ip address + 子网掩码；

② 开 SSH 服务：stelnet server enable + ssh server enable；

③ 生密钥对：public-key local create rsa；

④ 建本地用户：local-user admin → password → service-type ssh → user-role；

⑤ 配 VTY 接口：user-interface vty 0 63 → authentication-mode scheme → protocol inbound ssh。

安全加固：禁用 Telnet（明文）、限制 SSH 源地址（ACL）、定期更换密码、启用 SSH 密钥认证（public-key local import）。

☑☑ 速记口诀：五步顺序要记住：IP → 开服务 → 密钥 → 用户 → VTY。stelnet 是 H3C 专属叫法。

六、综合 & 场景题（高频应急/面试加分）

☒☒ 技术支持岗的面试亮点：应急排障思路、上线 checklist、诚实回答技巧。

[6-1] 综合

△☒ 为什么考这题：技术支持的核心场景题，面试官就是想看看你的排查思路是否系统。

☒ 问题：凌晨 3 点客户打电话说云平台某 VM 访问不了了，你怎么排查？

☒ 人话解释：按由近及远、由下至上的原则，每一层都要排除：

第 1 层：VM 内部（先登录 VM，登不上就看宿主）

- IP 还在吗？`ip a / ifconfig`
- 能 ping 通网关吗？`ping 192.168.10.1`
- 防火墙开了吗？`iptables / firewalld`
- 路由表对吗？`ip route`

第 2 层：物理交换机

- 接的端口是通的吗？`display interface GigabitEthernet x/x/x`
- VLAN 对吗？`display vlan`
- Trunk 口有没有漏 VLAN？
- STP 有没有把这个口阻塞了？`display stp brief`

第 3 层：上层网络

- 网关/路由能访问吗？ping 网关
- ACL 有没有挡？
- OSPF 邻居还在吗？`display ospf peer`

第 4 层：云平台层（VMware vCenter / OpenStack）

- VM 状态正常运行吗？
- vSwitch/Neutron 网络配置对吗？

关键：每一步留证据（截图、ping 输出、命令结果），方便写事故报告。

按由近及远、由下至上的原则，每一层都要排除：

第 1 层：VM 内部（先登录 VM，登不上就看宿主）

- IP 还在吗？`ip a / ifconfig`
- 能 ping 通网关吗？`ping 192.168.10.1`
- 防火墙开了吗？`iptables / firewalld`
- 路由表对吗？`ip route`

第 2 层：物理交换机

- 接的端口是通的吗？`display interface GigabitEthernet x/x/x`
- VLAN 对吗？`display vlan`
- Trunk 口有没有漏 VLAN？
- STP 有没有把这个口阻塞了？`display stp brief`

第 3 层：上层网络

- 网关/路由能访问吗？ping 网关
- ACL 有没有挡？
- OSPF 邻居还在吗？display ospf peer

第 4 层：云平台层（VMware vCenter / OpenStack）

- VM 状态正常运行吗？
- vSwitch/Neutron 网络配置对吗？

关键：每一步留证据（截图、ping 输出、命令结果），方便写事故报告。

☑ 面试标准答案：

系统排查思路（4 层递进）：

L1 - VM 内部：IP/路由/防火墙/进程状态（systemctl status）；

L2 - 虚拟化层：vSphere 看 VM Network 适配器 + vSwitch；OpenStack 看 Neutron 端口绑定；

L3 - 物理网络：H3C display interface 看端口 UP/DOWN/VLAN → display stp brief 看 STP → display arp 看 MAC 地址漂移 → ACL/安全策略；

L4 - 路由/上层：display ospf peer 邻居状态 → traceroute 逐跳定位 → BGP（如有）。

关键心态：别急着下结论，每一步确认排除后再下一步；留证据；先救服务再定位根因（比如先把 VM 切到备用宿主上恢复业务，再慢慢查根因）。

☑☑ 速记口诀：面试时主动说「先恢复业务再定位根因」，体现生产意识。

[6-2] 综合

△☑ 为什么考这题：JD 明确要求 MGR 高可用部署，故障处理必问。

☑ 问题：MGR 集群有一个节点掉线了（SECONDARY 挂了），怎么处理？

☑ 人话解释：想象你那个三人投票小组（3 节点 MGR），有一个人请假了，咋办？

第 1 步：先看影响——组长（PRIMARY）还在吗？另外两个组员能投票吗？

- 3 节点掉 1 个还剩 2 个，多数派（2/3）还能工作，业务不受影响
- 但如果掉的是 PRIMARY，会自动从剩下两个 SECONDARY 里选新 PRIMARY

第 2 步：登录掉线的那台机器，看 MySQL error.log 最后 200 行——常见原因：

- 磁盘满了（df -h，undo/redo 日志爆了）
- 内存不够 OOMKilled（dmesg -T | grep oom）
- 网络抖动（丢包超过 group_replication_message_cache_size 丢事务）
- 大事务阻塞（一个 SECONDARY 还没追上来就有新事务）

第 3 步：修好了让它自动追：

```
`mysql> START GROUP_REPLICATION;`
```

然后看 `SELECT \* FROM performance\_schema.replication\_group\_members;`，等它变成 ONLINE

第 4 步：如果追不上（有 errant transaction，事务冲突了）

→ STOP + RESET SLAVE ALL + 去 PRIMARY 上做一份备份再重加入

第 5 步：事后复盘——为啥掉？加告警（Prometheus + exporter）、调大磁盘/内存、优化大事务。

想象你那个三人投票小组（3 节点 MGR），有一个人请假了，咋办？

第 1 步：先看影响——组长（PRIMARY）还在吗？另外两个组员能投票吗？

- 3 节点掉 1 个还剩 2 个，多数派（2/3）还能工作，业务不受影响
- 但如果掉的是 PRIMARY，会自动从剩下两个 SECONDARY 里选新 PRIMARY

第 2 步：登录掉线的那台机器，看 MySQL error.log 最后 200 行——常见原因：

- 磁盘满了（df -h，undo/redo 日志爆了）
- 内存不够 OOMKilled（dmesg -T | grep oom）
- 网络抖动（丢包超过 group_replication_message_cache_size 丢事务）
- 大事务阻塞（一个 SECONDARY 还没追上来就有新事务）

第 3 步：修好了让它自动追：

```
`mysql> START GROUP_REPLICATION;`
```

然后看 `SELECT \* FROM performance\_schema.replication\_group\_members;`，等它变成 ONLINE

第 4 步：如果追不上（有 errant transaction，事务冲突了）

→ STOP + RESET SLAVE ALL + 去 PRIMARY 上做一份备份再重加入

第 5 步：事后复盘——为啥掉？加告警（Prometheus + exporter）、调大磁盘/内存、优化大事务。

☑ 面试标准答案：

MGR 节点掉线处理流程：

① 确认集群状态：SELECT * FROM replication_group_members → 看 PRIMARY 是否正常，多数派是否还在；

② 查掉线原因：看 error.log、磁盘(df -h)、内存(free -h)、网络(ping 其它节点)、OOM(dmesg)；

③ 重新加入集群：

- 直接重加：STOP GROUP_REPLICATION → START GROUP_REPLICATION → 等自动追平；

- 有事务冲突（errant transaction）：RESET SLAVE ALL → 从 PRIMARY 做 mysqldump → 导入本节点 → START GROUP_REPLICATION；

- 网络脑裂：人工裁决（选数据最新的节点），把其它节点 RESET 后重加；

④ 预防：奇数节点（≥3）、同机房低延迟、磁盘 70% 告警、慢查询优化大事务。

☑☑ 速记口诀：记住：奇数节点、多数派、自动选举、errant transaction 是高频坑。

[6-3] 综合

△☑ 为什么考这题：JD 职责里写了「负责产品上线前的业务及功能测试」，面试官可能会问。

☑ 问题：新系统上线前，技术支持要做哪些测试和检查？

☒ 人话解释：想象你家新装修完要入住，怎么检查？

业务功能测试：每个房间的灯开关都试一下（核心业务链路走一遍），新装修的特殊设计（新功能）重点试；

- 性能压测：洗衣机+空调+烤箱一起开会不会跳闸？（JMeter

压测对比基线，CPU/内存/DB/网络不超阈值）；

- 兼容测试：不同手机能不能连 WiFi？不同浏览器页面正常？不同数据库版本？；
- 安全检查：门锁好没？窗户关了？（弱口令、越权访问、SQL 注入、XSS）；
- 灰度发布：先让客厅（灰度集群）通通电观察 1-2 小时没事再让全楼通电；
- 回滚预案：电表总闸在哪？开关的旧灯泡存哪？（数据库回滚脚本、镜像/二进制包备份、VM 快照）。

想象你家新装修完要入住，怎么检查？

- 业务功能测试：每个房间的灯开关都试一下（核心业务链路走一遍），新装修的特殊设计（新功能）重点试；

- 性能压测：洗衣机+空调+烤箱一起开会不会跳闸？（JMeter

压测对比基线，CPU/内存/DB/网络不超阈值）；

- 兼容测试：不同手机能不能连 WiFi？不同浏览器页面正常？不同数据库版本？；
- 安全检查：门锁好没？窗户关了？（弱口令、越权访问、SQL 注入、XSS）；
- 灰度发布：先让客厅（灰度集群）通通电观察 1-2 小时没事再让全楼通电；
- 回滚预案：电表总闸在哪？开关的旧灯泡存哪？（数据库回滚脚本、镜像/二进制包备份、VM 快照）。

☒ 面试标准答案：

上线前 checklist（技术支持视角）：

- ① 功能回归：核心业务链路 + 新 Feature + 边界 case；
- ② 性能基线：JMeter/ab 压测 P95 响应时间 + 并发数，对比旧版本 CPU/内存/DB QPS/IOPS；
- ③ 兼容性：浏览器（Chrome/Edge/Safari）、OS、数据库版本（MySQL 8.0 vs 5.7）、协议；
- ④ 安全自检：弱口令扫描、越权测试、SQL 注入/XSS 基础检查；
- ⑤ 灰度策略：先灰度 5% 流量/灰度集群观察 1-2h → 逐步放量；
- ⑥ 回滚预案：
 - DB：准备回滚脚本 + 上线前备份；
 - 应用：镜像 tag 保留上一个版本（docker tag old version）；
 - VM：快照/Clone；
- ⑦ 监控告警：确保新服务的 metrics 接入 Prometheus/Zabbix，告警规则配好；
- ⑧ 知识库更新：上线后把新架构/新特性记入内部 wiki。

☒☒ 速记口诀：回滚预案一定要有，面试官很看重生产风险意识。

[6-4] 综合

⚠☒ 为什么考这题：面试常见诚实回答的问题，不能吹但要展示方法论。

☒ 问题：JD 要求「具备 TB 级数据量使用经验者优先」，你没做过真 TB 级，怎么说？

☒ 人话解释：诚实说没做过 TB 级，但你理解它的挑战，并且有小规模经验可复用：

"确实没独立维护过 TB 级集群，但我了解 TB 级的核心挑战：

- ① 存储：单表超过 5000 万行必须分表，不然 B+ 树高度超过 3 层（默认 16KB 页，3 层能存约 2000 万，4 层就慢了）；

- ② 分库分表：分片键选 user_id / order_id，hash 取模分成 16/32 个分片，中间件用 ProxySQL 或 ShardingSphere-JDBC；
- ③ 冷热分离：近 6 个月数据在 MySQL（热），老数据归档到 PG/ClickHouse/HDFS（冷）；
- ④ 带宽和 IOPS：TB 级全表备份要几小时，所以用部分备份 + 增量备份 + binlog 归档，存储用 NVMe SSD 扛 IOPS；

我之前做过千万级单表的分表和索引优化，方法论是通的，上手 TB 级不会有问题。"

诚实说没做过 TB 级，但你理解它的挑战，并且有小规模经验可复用：

"确实没独立维护过 TB 级集群，但我了解 TB 级的核心挑战：

- ① 存储：单表超过 5000 万行必须分表，不然 B+ 树高度超过 3 层（默认 16KB 页，3 层能存约 2000 万，4 层就慢了）；
- ② 分库分表：分片键选 user_id / order_id，hash 取模分成 16/32 个分片，中间件用 ProxySQL 或 ShardingSphere-JDBC；
- ③ 冷热分离：近 6 个月数据在 MySQL（热），老数据归档到 PG/ClickHouse/HDFS（冷）；
- ④ 带宽和 IOPS：TB 级全表备份要几小时，所以用部分备份 + 增量备份 + binlog 归档，存储用 NVMe SSD 扛 IOPS；

我之前做过千万级单表的分表和索引优化，方法论是通的，上手 TB 级不会有问题。"

☒ 面试标准答案：

核心策略：承认 + 展示方法论 + 关联自己的经验。

"TB 级和十亿级的核心技术挑战是相同的：分库分表（ProxySQL/ShardingSphere + 合适的分片键 hash/range）、冷热分离（近 6 月在 MySQL，历史归档 ClickHouse）、高可用（MGR 集群 + ProxySQL 入口层）、备份恢复（xtrabackup 部分备份 + binlog 归档）、慢查询治理（Pt-query-digest 每周巡检）。我做过单表 2000 万级的索引优化和分表改造，方法论一致，规模扩大 100 倍主要是硬件成本增加，技术方案本身我有信心在一周内上手。"

☒☒ 速记口诀：别说"我做过 TB 级"，面试官一问细节就穿帮。诚实 + 方法论 + 小案例。

七、速记口诀表（考前 5 分钟必看）

☒☒ 下面是所有高频考点的压缩版关键词，面试路上过一遍即可唤醒记忆。

考点	速记口诀
MySQL 引擎	InnoDB=事务+行锁+聚簇索引+redo/undo；MyISAM=表锁+无事务，淘汰
聚簇索引	主键索引=聚簇（叶子存整行）；二级索引叶子存主键值→回表；覆盖索引=查啥建啥
慢查询排查	开 slow_query_log → pt-query-digest 分析 → EXPLAIN type/key/rows → 加索引/改SQL
MGR 集群	3节点+奇数+Paxos 投票；单主最稳；START GROUP_REPLICATION；errant transaction 重加
PG vs MySQL	PG=复杂SQL/地理/JSONB/MVCC真多版本；MySQL=高并发简单读写；信创项目用PG/达梦
达梦	信创国产；dminit 初始化；dmrman 备份；DTS 迁移；原理=Oracle/PG
Mycat/ProxySQL	分库分表+读写分离；ProxySQL=轻量+SQLite配置；Mycat=老牌+功能全但坑多
Docker	镜像=分层缓存；Dockerfile 多阶段构建；先 COPY 依赖后源码；cmd vs entrypoint
Compose vs K8s	Compose=单机YAML编排；K8s=集群(Pod/Service/Deployment/Ingress)
K8s 组件	控制面:apiserver+etcd+scheduler+controller；数据面:kubelet+proxy+containerd
VMware/OpenStack	KVM=底座；VMware=商业付费 vSphere；OpenStack=开源 IaaS(Nova/Cinder/Neutron)
Zabbix/Prometheus/Grafana	Zabbix=Agent推送+模板；Prometheus=Pull模型+TSDB+PromQL；Grafana=可视化面板
Shell 备份脚本	mysqldump --single-transaction → gzip → find -mtime +7 -delete → crontab 0 2 * * *
CPU 飙高排查	top找进程 → pidstat找线程 → perf抓函数 → jstack/arthas看Java → 常见死循环/GC风暴
TCP 三次握手	SYN → SYN+ACK → ACK；双向确认收发能力；TIME_WAIT=2MSL防迟到报文
VLAN	Access=单VLAN接终端；Trunk=多VLAN打802.1Q标签接交换机；display vlan 查看
STP	根桥=最小Priority+MAC；RSTP收敛1-2s；MSTP多实例负载均衡；企业用RSTP/MSTP
OSPF vs RIP	OSPF=链路状态+Dijkstra+Area 0骨干；RIP=距离矢量+跳数15+收敛慢淘汰
H3C SSH 五步	① 配管理IP ② stelnet server enable ③ public-key local create rsa ④ local-user ⑤ vty 配置
VM 访问不了排查	VM内部IP/防火墙 → 物理交换机VLAN/STP → 路由/OSPF邻居 → 云平台层
MGR 节点掉线处理	查 error.log/磁盘/OOM → STOP+START GROUP_REPLICATION → errant 事务 RESET 重加
上线前 checklist	功能回归+性能压测+兼容+安全+灰度+回滚预案(DB脚本/镜像/tag/快照)
诚实答没经验	承认没 TB 级 → 讲方法论(分库分表/冷热分离/MGR) → 关联自己小规模经验

☒ 面试临场 Checklist（进考场前默念）

☒ 自我介绍准备好：名字 + 几年经验 + 做过啥 + 为什么来云趣

☒ 主动说 MGR 集群部署经验（JD 明确提了）

- ☒ 主动说 MySQL 慢查询调优的真实案例（数字说话：调优前 500ms → 调优后 50ms）
- ☒ 达梦数据库：承认没真用过 + 原理和 Oracle/PG 通 + 上手一周能接管
- ☒ 没做过 TB 级：承认 + 讲方法论 + 关联自己千万级经验
- ☒ 网络：VLAN/Trunk/STP/OSPF 这四个概念要能串起来讲
- ☒ 应急排障思路一定 由近及远 + 先救业务再定位根因
- ☒ 反问环节可以问：公司现在用的是哪个 MGR 版本？达梦迁移进度？
- ☒ 穿正装（至少休闲商务，技术岗不用西装但别穿拖鞋）
- ☒ 提前 15 分钟到，路上别刷手机，过最后那页口诀表